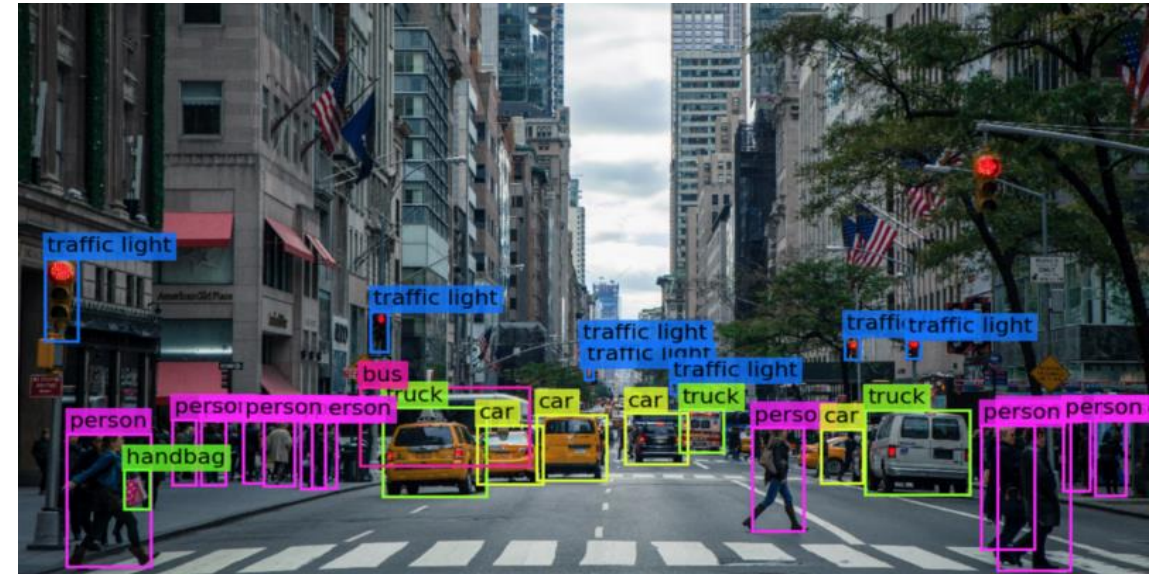


Deep Learning

Convolutional neural networks

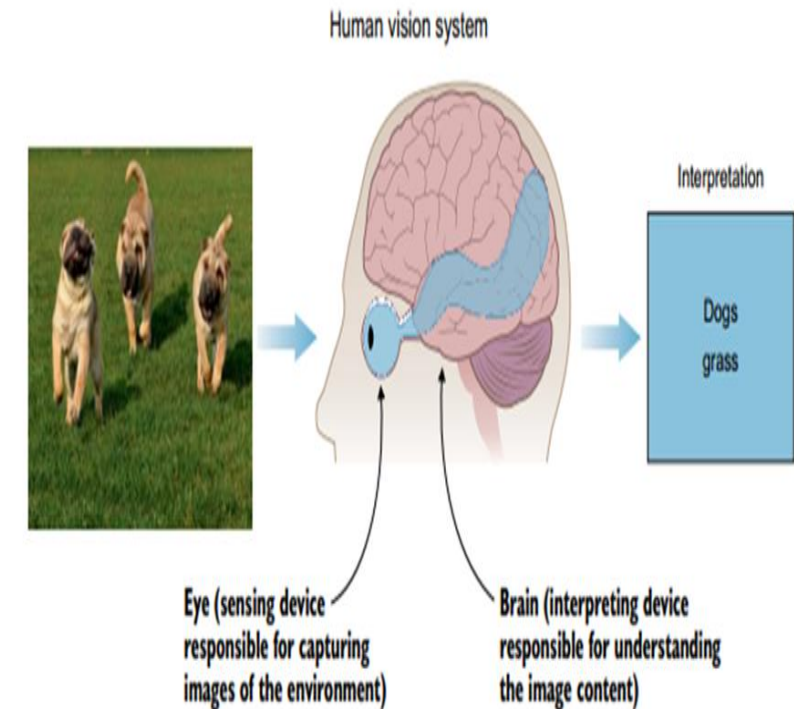
Computer vision

- Computer vision is a field of artificial intelligence (AI) that enables computers and systems to derive meaningful information from digital images, videos and other visual inputs — and take actions or make recommendations based on that information.



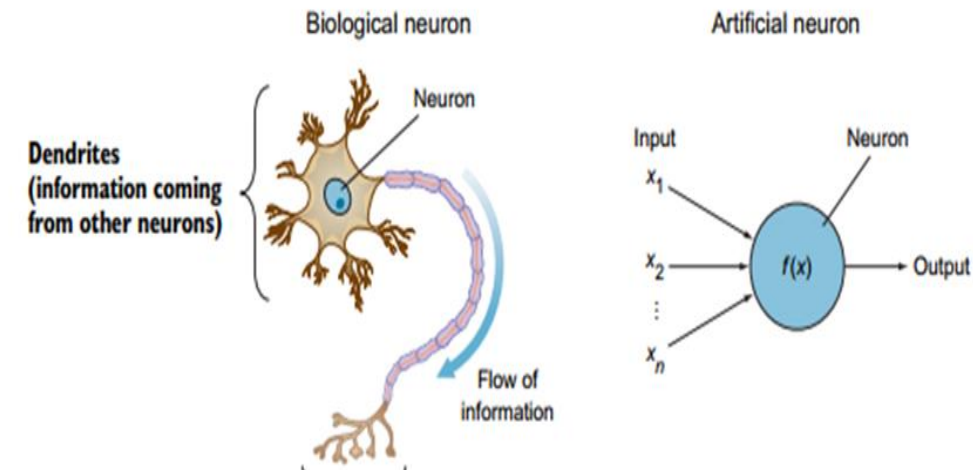
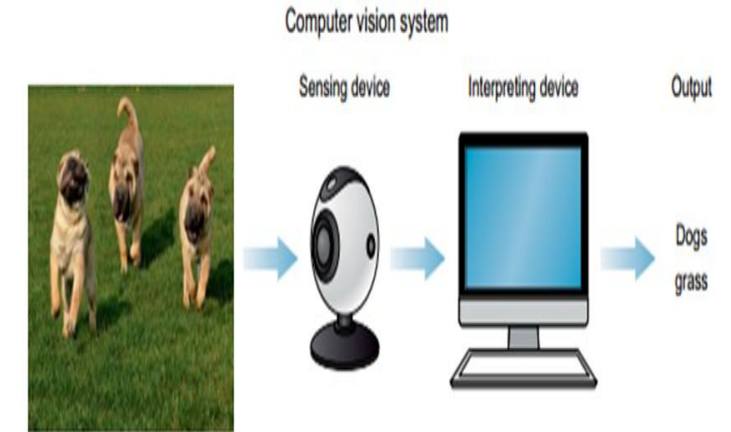
Human Vision Systems

- At the highest level, vision systems are pretty much the same for humans, animals, insects.
- They consist of a sensor or an eye to capture the image and a brain to process and interpret the image.
- The system then outputs a prediction of the image components based on the data extracted from the image.



AI Vision Systems

- Scientists were inspired by the human vision system to simulate the human vision system, we need the same two main components:
 - a sensing device to simulate the function of the eye
 - a powerful algorithm to mimic the brain function in interpreting and classifying image content.
- *Scientists were inspired by how our brains work and tried to reverse engineer the central nervous system to get some insight on how to build an artificial brain. Thus, artificial neural networks (ANNs) were born.*



Applications of computer vision

- Image classification

Lung cancer diagnosis

Traffic sign recognition

- Object detection and localization

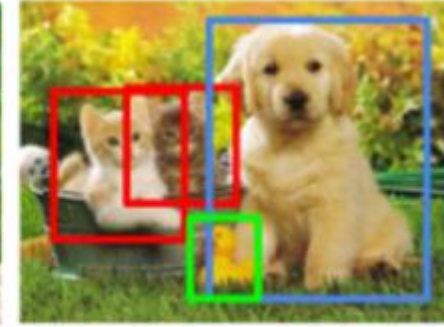
- Face recognition

Classification



Cat

Detection



Cat, Duck



How computers see images?

- A computer sees images as matrices of values. The values represent the intensity of the pixels across the colour spectrum.
- For example, grayscale images range between pixel values of 0 for black and 255 for white.

What we see



What computers see

08	02	22	97	38	15	00	40	00	75	04	05	07	78	52	12	50	77	91	08
49	49	99	40	17	81	18	57	60	87	17	40	98	43	69	46	04	56	62	00
81	49	31	73	55	79	14	29	93	71	40	67	53	99	30	03	49	13	36	65
52	90	95	23	04	60	11	42	69	24	48	56	01	32	54	71	37	02	34	91
22	31	14	71	51	67	43	59	41	92	34	54	22	40	40	28	44	33	13	80
24	47	32	60	99	03	45	02	44	75	33	53	78	36	64	20	35	09	12	80
32	98	81	28	64	23	67	10	26	38	40	67	59	54	70	66	18	38	64	70
47	24	20	68	02	62	12	20	95	63	94	39	63	04	49	91	44	49	94	21
24	55	58	05	66	73	99	26	97	17	78	78	94	83	14	88	34	89	63	72
21	36	23	09	75	00	74	44	20	45	35	14	00	41	33	97	34	31	33	95
78	17	53	28	22	75	31	67	15	94	03	80	04	42	16	14	09	53	56	92
16	39	05	42	96	35	31	47	55	58	88	24	00	17	54	24	34	29	85	57
84	56	00	48	35	71	89	07	05	44	44	37	44	40	21	58	51	54	17	58
19	80	61	68	05	94	47	49	28	73	92	13	86	52	17	77	04	09	55	40
04	52	08	83	97	35	99	14	07	97	57	32	16	26	26	79	33	27	98	44
04	36	68	81	57	62	20	72	03	16	33	67	46	55	12	32	43	93	53	69
04	42	14	73	38	25	39	11	24	94	72	18	06	46	29	32	40	62	74	36
20	49	34	41	72	30	23	88	34	62	99	69	82	47	59	85	74	04	34	24
20	23	35	29	78	31	90	01	74	31	49	71	48	86	81	14	23	57	05	54
01	70	54	71	83	51	54	49	16	92	33	48	61	43	51	01	89	19	67	48

- In colour images have three channels (red, green, and blue). Images are represented by three matrices: one represents the intensity of red in the pixel, one represents green, and one represents blue.

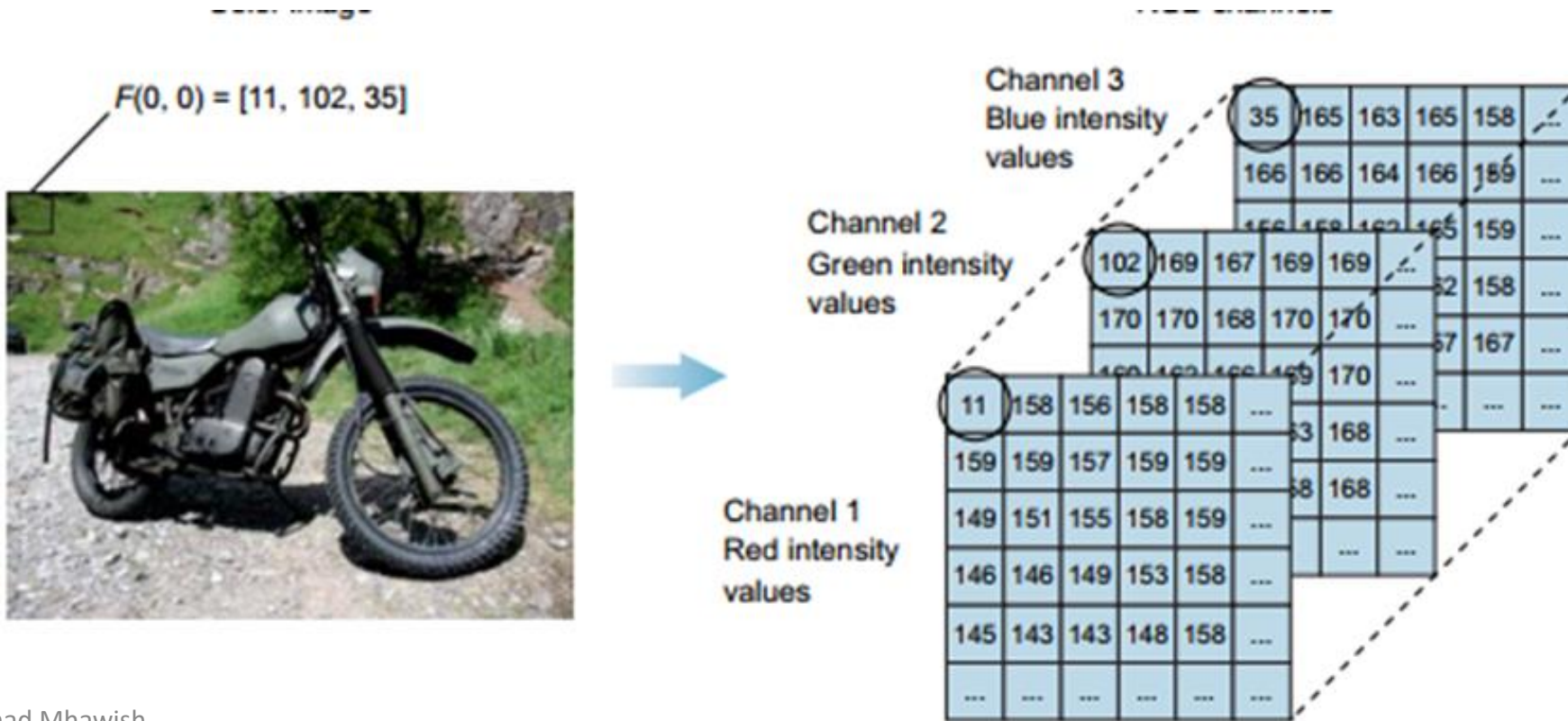
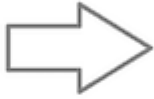


Image classification using MLP (fully connected network)

1	1	0
4	2	1
0	2	1



1
1
0
4
2
1
0
2
1

Visualizing input vectors

To help visualize the flattened input vector, let's look at a much smaller matrix (4, 4):

1	0	0	0
1	1	1	0
1	1	0	1
1	0	1	1

The input (x) is a flattened vector with the dimensions (1, 16):

x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	x_9	x_{10}	x_{11}	x_{12}	x_{13}	x_{14}	x_{15}	x_{16}
Row 1				Row 2				Row 3				Row 4			

So, if we have pixel values of 0 for black and 255 for white, the input vector will be as follows:

Input = [0, 255, 255, 255, 0, 0, 0, 255, 0, 0, 255, 0, 0, 255, 0, 0]

Image classification using MLP (fully connected network)

Input Layer

Here is how we flatten an input image in Keras:

```
from keras.models import Sequential
from keras.layers import Flatten
```

Defines the model

```
model = Sequential()
model.add( Flatten(input_shape = (28,28) ) )
```

As before, imports the Keras library

Imports a layer called Flatten to convert the image matrix into a vector

Adds the Flatten layer, also known as the input layer

Hidden layers

```
from keras.layers import Dense
```

Imports the Dense layer

```
model.add(Dense(512, activation = 'relu'))
model.add(Dense(512, activation = 'relu'))
```

Adds two Dense layers with 512 nodes each

Output layer

```
model.add(Dense(10, activation = 'softmax'))
```

The Flatten layer in Keras handles this process for us. It takes the 2D image matrix input and converts it into a 1D vector. Note that the Flatten layer must be supplied a parameter value of the shape of the input image.

```

Imports the Keras library
from keras.models import Sequential
from keras.layers import Flatten, Dense

Imports a Flatten layer
to convert the image
matrix into a vector

Adds the
Flatten
layer
model = Sequential()
model.add( Flatten(input_shape = (28,28) ))

Defines the neural
network architecture

model.add(Dense(512, activation = 'relu'))
model.add(Dense(512, activation = 'relu'))

Adds 2 hidden layers with 512 nodes
each. Using the ReLU activation
function is recommended in
hidden layers.

model.add(Dense(10, activation = 'softmax'))
model.summary()

Prints a summary
of the model
architecture

Adds 1 output Dense layer with
10 nodes. Using the softmax
activation function is recommended
in the output layer for multiclass
classification problems.

```

Layer (type)	Output Shape	Param #
Flatten_1 (Flatten)	(None, 784)	0
dense_1 (Dense)	(None, 512)	401920
dense_2 (Dense)	(None, 512)	262656
dense_3 (Dense)	(None, 10)	5130
Total params: 669,706		
Trainable params: 669,706		
Non-trainable params: 0		

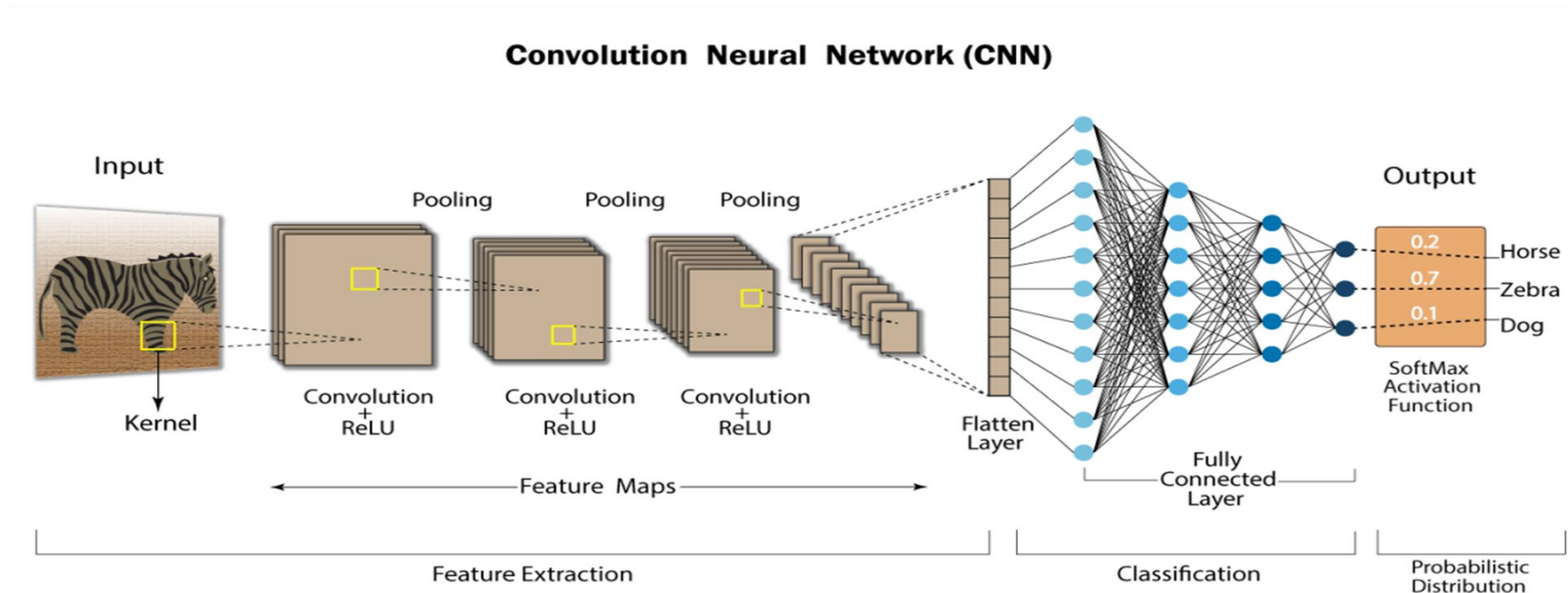
- Parameters after the flatten layer = 0, because this layer only flattens the image to a vector for feeding into the input layer. The weights haven't been added yet.
- Parameters after layer 1 = (784 nodes in input layer) $\times$ (512 in hidden layer 1) + (512 connections to biases) = 401,920.
- Parameters after layer 2 = (512 nodes in hidden layer 1) $\times$ (512 in hidden layer 2) + (512 connections to biases) = 262,656.
- Parameters after layer 3 = (512 nodes in hidden layer 2) $\times$ (10 in output layer) + (10 connections to biases) = 5,130.
- Total params in the network = 401,920 + 262,656 + 5,130 = 669,706.

- Disadvantage of using Dense (fully connected layers network in Image classification)
- **SPATIAL FEATURE LOSS**
- Huge number of trainable parameters
 - No enough data
 - High computation effort

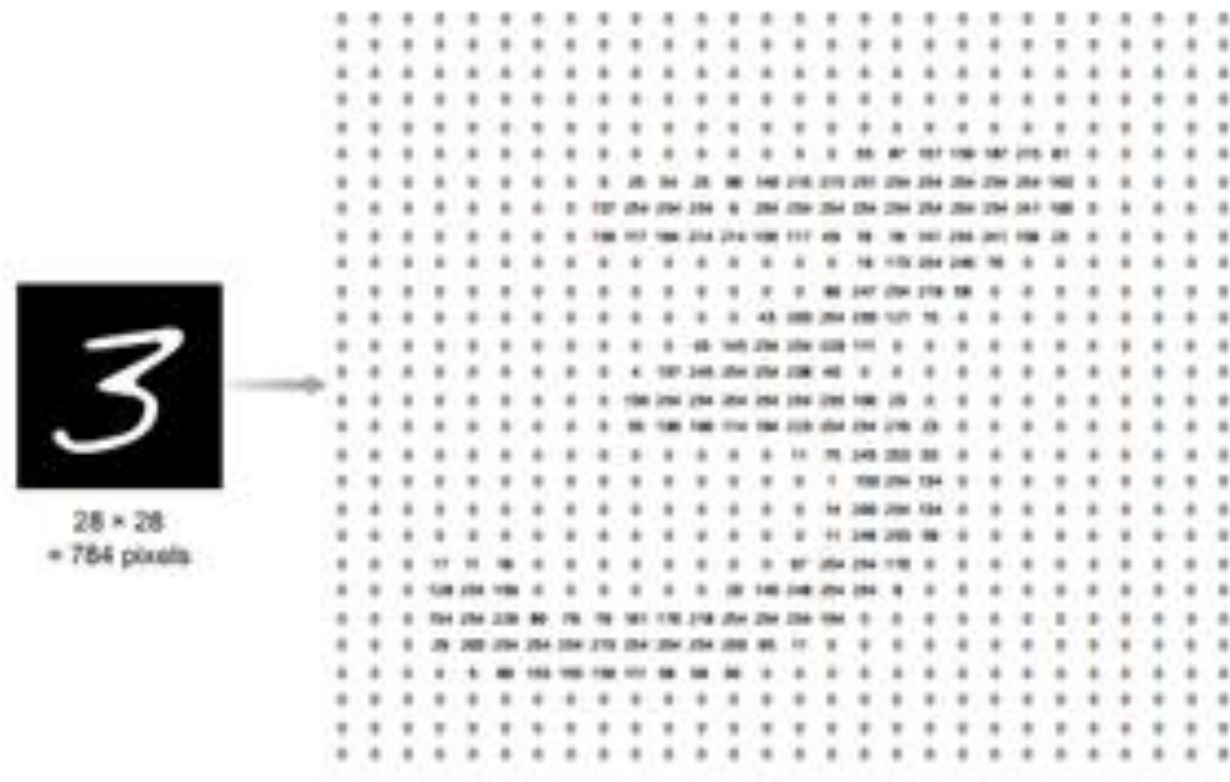
Convolutional neural networks

- A convolutional neural network (CNN) is a type of artificial neural network (Deep Learning) used in image recognition and processing that is specifically designed to process pixel data.

- Input
- Convolution Layers
 - CONV Layer
 - ReLU Layer
 - Pool Layer
- Fully Connected Layer

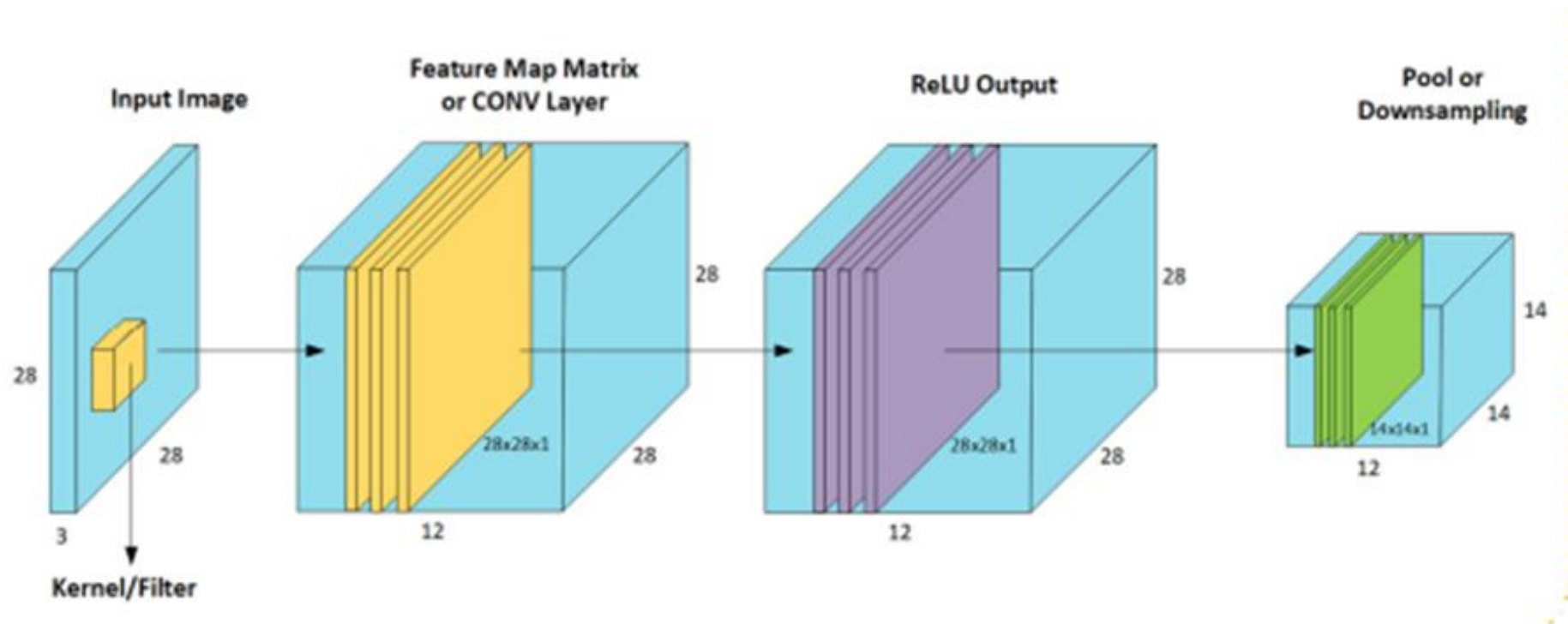


1: Input layer



2: Convolutional layers

- Convolutional layers act like a feature finder window that slides over the image pixel by pixel to extract meaningful features that identify the objects in the image.



2.1: CONV layer (the output :Features map)

- In mathematics, convolution is the operation of two functions to produce a third modified function.

In the context of CNNs

- The first function is the input image
- The second function is the convolutional filter (kernel).
- The third modified function is the output is called a Feature Map.

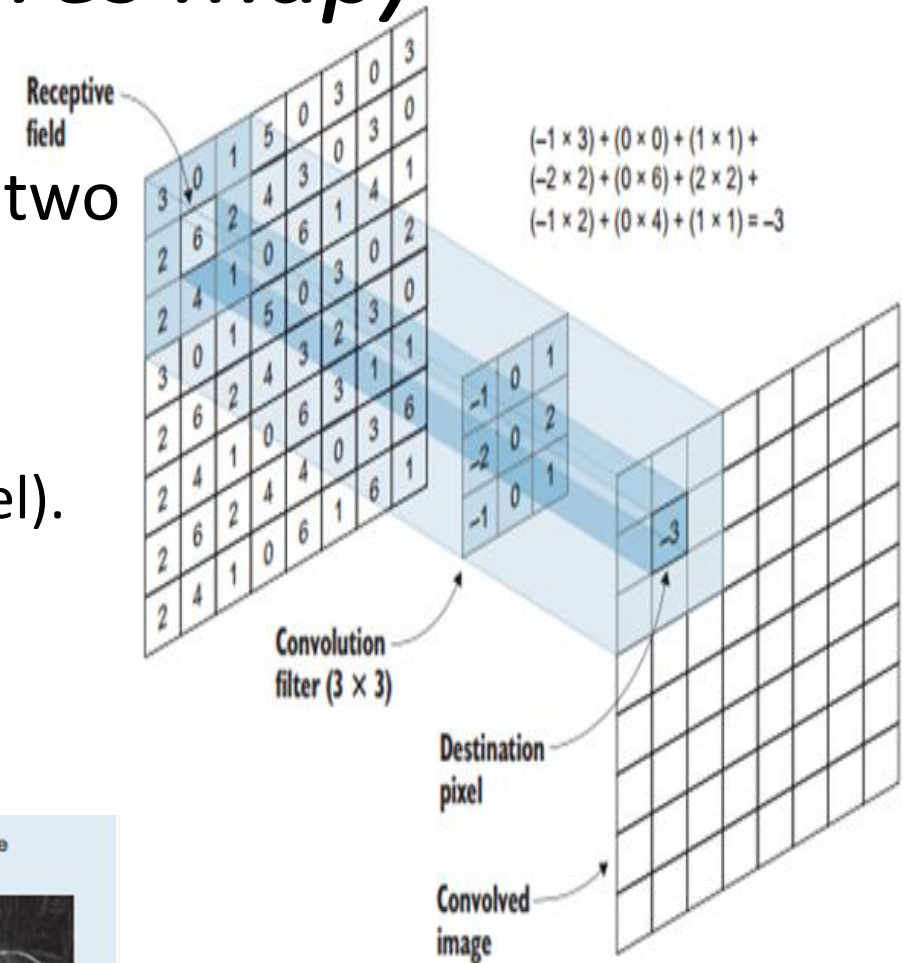
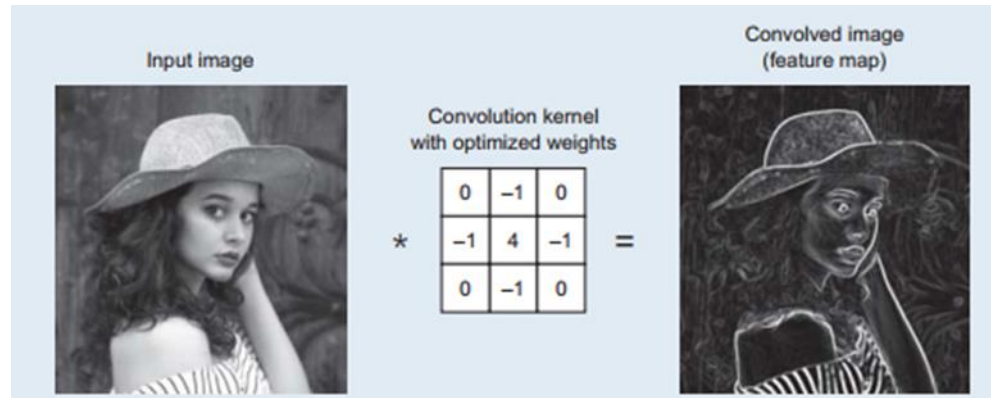
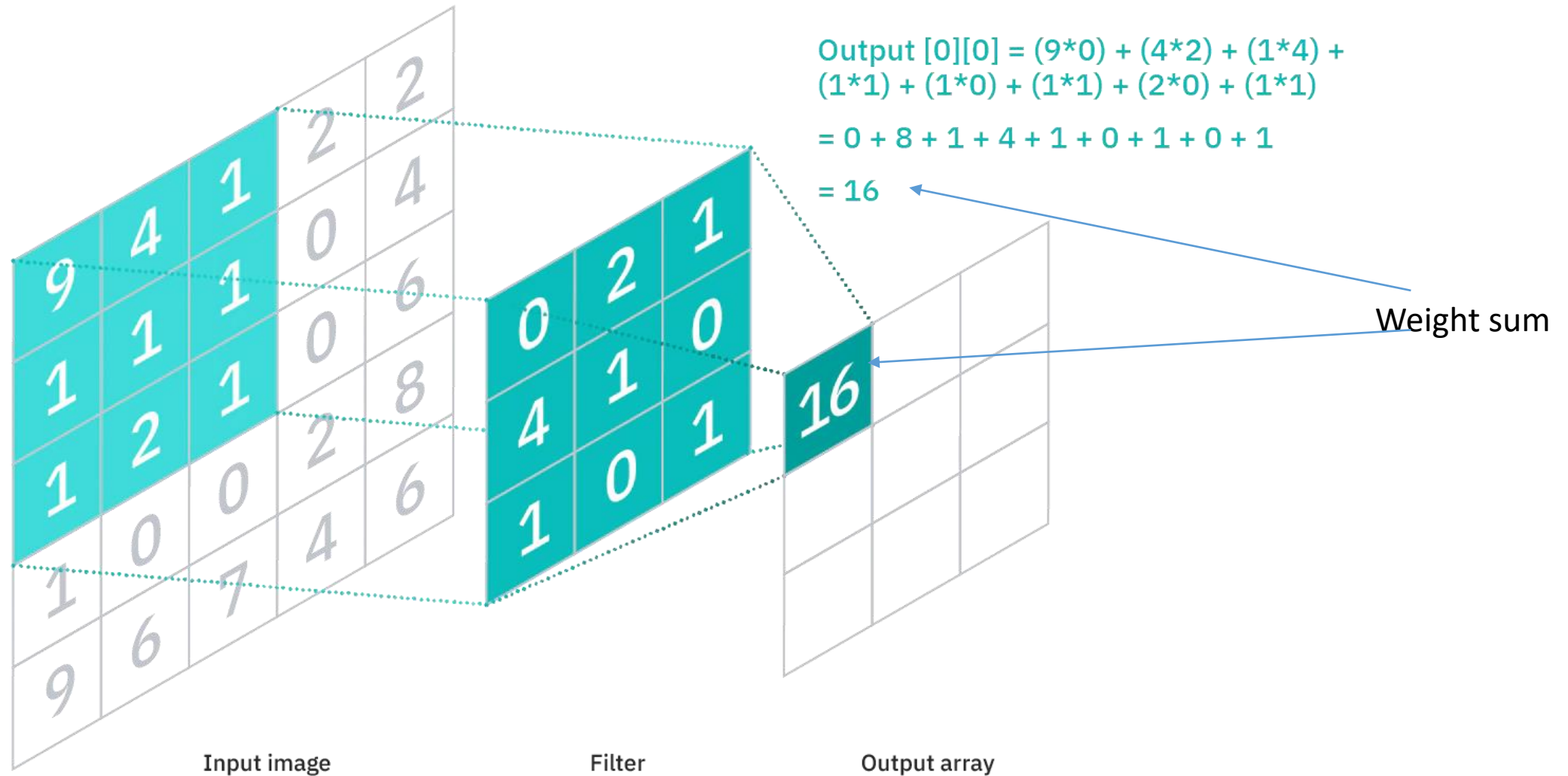


Image Features are simply interesting areas of an image.

Examples:

- Edges
- Colors
- Patterns/Shapes





$$\text{weighted sum} = x_1 \cdot w_1 + x_2 \cdot w_2 + x_3 \cdot w_3 + \dots + x_n \cdot w_n + b$$

2.1: CONV layer (Feature map) contd.

- The following parameters to control the size of our feature maps.
 - **Kernel (Filter) Size** ($K \times K$) or ($f \times f$) the size of the convolutional filter matrix.
 - **Depth**: the number of convolutional filters in each layer. This represents the depth of its output. Each filter learns different feature maps that are activated in the presence of different image features (edges, patterns, colours, layouts)
 - **Strides**: The amount by which the filter slides over the image. For example, to slide the convolution filter one pixel at a time, the strides value is 1.

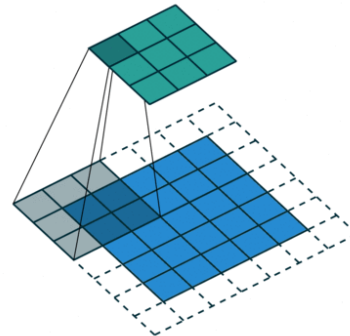
1x0	0x1	1x0	0	1
1x1	0x0	0x-1	1	1
0x0	1x1	1x0	0	0
1	0	0	1	0
0	0	1	1	0

1	0x0	1x1	0x0	1
1	0x1	0x0	1x-1	1
0	1x0	1x1	0	0
1	0	0	1	0
0	0	1	1	0

1	0	1x0	0x1	1x0
1	0	0x1	1x0	1x-1
0	1	1x0	0x1	0x1
1	0	0	1	0
0	0	1	1	0

1	0	1	0	1
1x0	0x1	0x0	1	1
0x1	1x0	1x-1	0	0
1x0	0x1	0x0	1	0
0	0	1	1	0

- **Padding**: Often called zero-padding because we add zeros around the border of an image . used to allow us to preserve the spatial size of the input volume so the input and output width and height are the same.



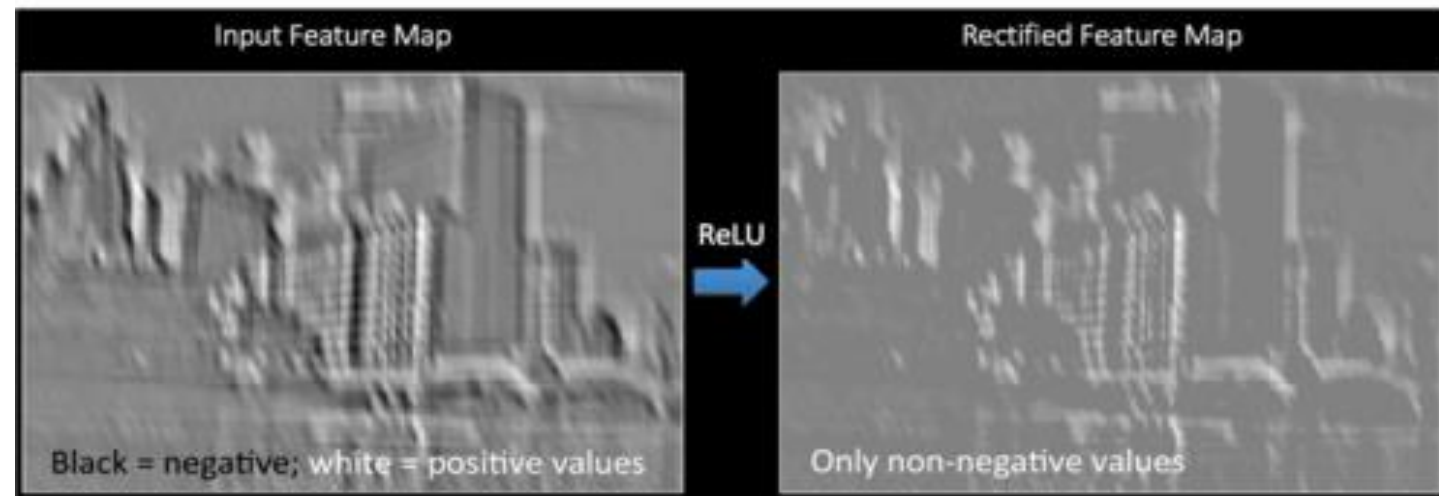
2.2: Activation Function (ReLU layer)

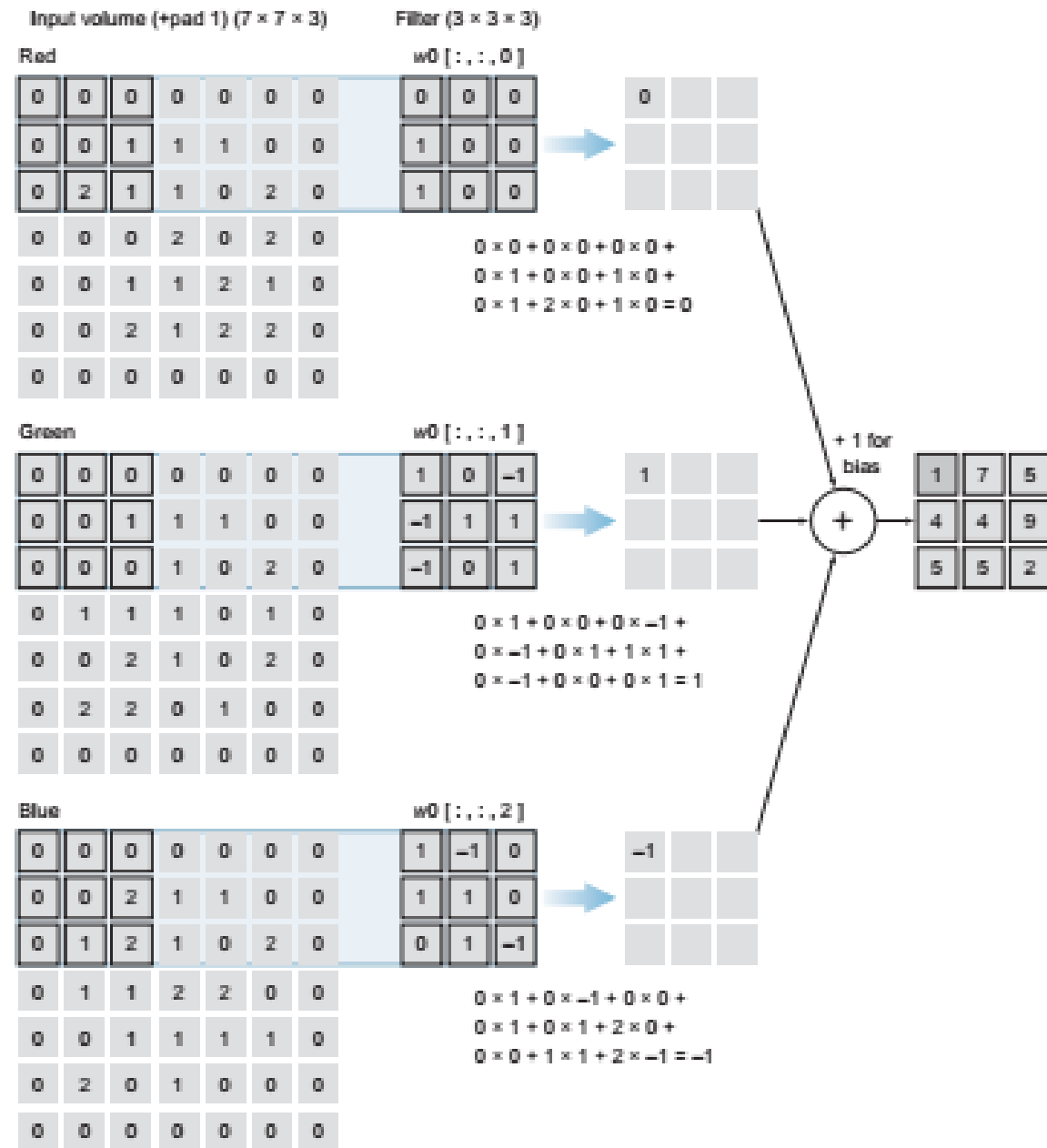
- An activation function is placed at the end of each perceptron to decide whether to activate this neuron.
- Without the activation function, we just have a linear function that produces a number, but no decision is made in this perceptron.
- For CNN, ReLU is the Activation Function of Choice
 - ReLU or Rectified Linear Unit simply changes all the negative values to 0 while leaving the positives values unchanged.

2	1	-5
2	0	4
-5	3	-1

Applying ReLU

2	1	0
2	0	4
0	3	0





Input volume (+pad 1) ($7 \times 7 \times 3$)

0	0	0	0	0	0	0
0	0	1	1	1	0	0
0	2	1	1	0	2	0
0	0	0	2	0	2	0
0	0	1	1	2	1	0
0	0	2	1	2	2	0
0	0	0	0	0	0	0
0	0	0	0	0	0	0
0	0	1	0	2	1	0
0	0	0	1	1	2	0
0	1	1	1	0	1	0
0	0	2	1	0	2	0
0	2	2	0	1	0	0
0	0	0	0	0	0	0
0	0	0	0	0	0	0
0	0	2	2	2	2	0
0	1	2	0	2	1	0
0	1	1	2	2	0	0
0	0	1	1	1	1	0
0	2	0	1	0	0	0
0	0	0	0	0	0	0

Filter 1

0	0	0
1	0	0
1	0	0

1	0	-1
-1	1	1
-1	0	1

Bias b_0 ($1 \times 1 \times 1$)

$b_0[:, :, 0]$
1

Filter 2

1	-1	1
-1	-1	1
-1	-1	1

1	-1	1
-1	0	-1
-1	1	1

Bias b_1 ($1 \times 1 \times 1$)

$b_1[:, :, 0]$
0

Output volume ($3 \times 3 \times 2$) $o[:, :, 0]$

1	7	5
4	4	9
5	5	2

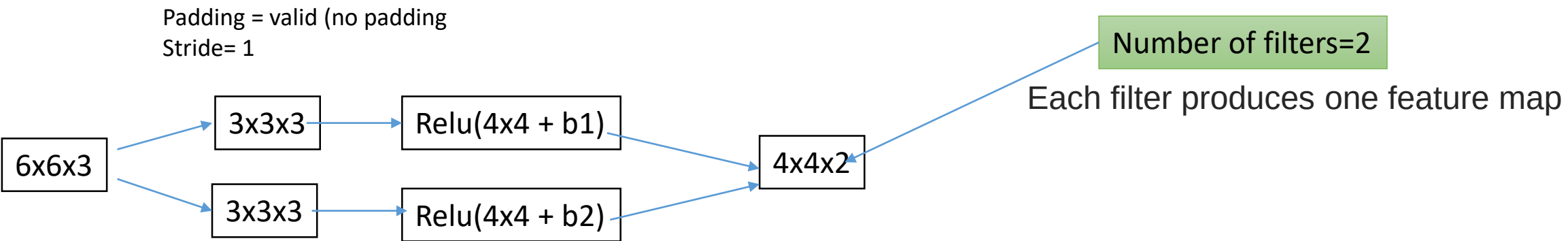
 $o[:, :, 1]$

4	4	1
6	6	1
4	2	-4

Note: X here is multiply

- $N \times N$ image size : $N \times N \times CH$... CH = channel (for image color =3)
- $F \times F$ filter size: $F \times F \times CH$... CH = channel (must be same of the input channel)
- The output = $n-f+1 \times n-f+1$

Ex: one filter : **Image** $6 \times 6 \times 3$ **filter** $3 \times 3 \times 3$ **Features map** 4×4 (where trainable parameter? Each item in filter is w
27 +1 (trainable parameters)



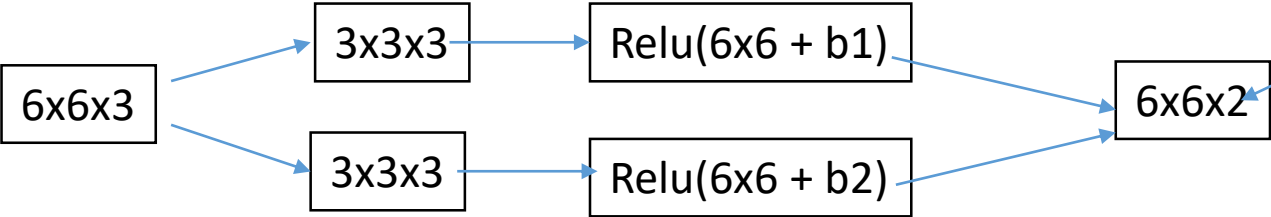
How many trainable parameters here?

Note: X here is multiply

Padding = same
Stride = 1
Number of filter =2

Features map

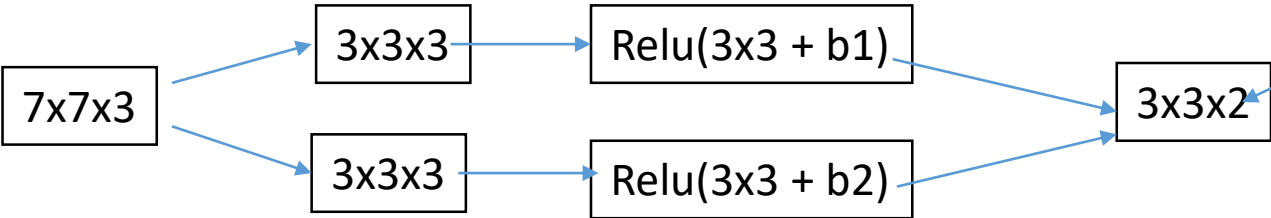
Number of filters=2



Padding = same
Stride = 2
Number of filter =2

Features map

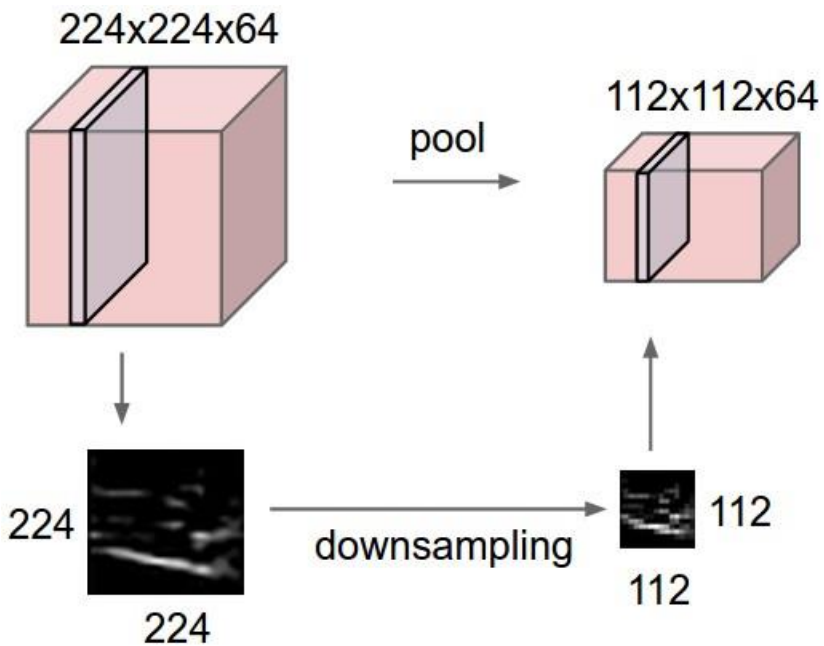
Number of filters=2



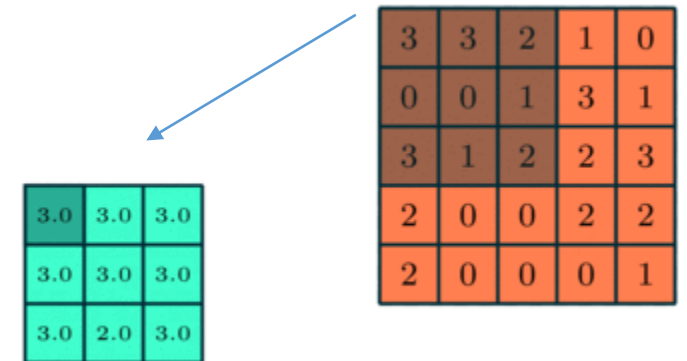
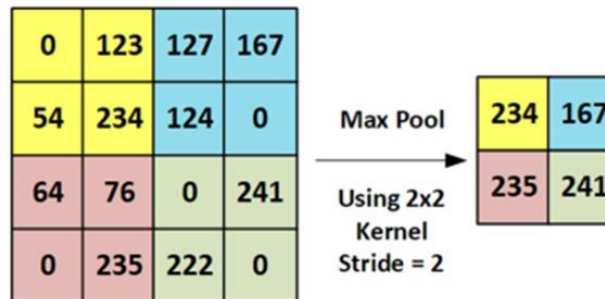
:
$$\text{output} = \frac{N + 2p - f}{S} + 1$$

2.3: Pooling layer

- Pooling, known as subsampling or down sampling
- simple process where we reduce the size or dimensionality of the Feature Map.
- The purpose of this reduction is to **reduce** the number of **parameters** needed to train, whilst **retaining the most important features**.
- There are 3 types of Pooling we can apply, *Max*, *Average* and *Sum*.



$$\text{output} = \frac{N - f(\text{pooling})}{S} + 1$$

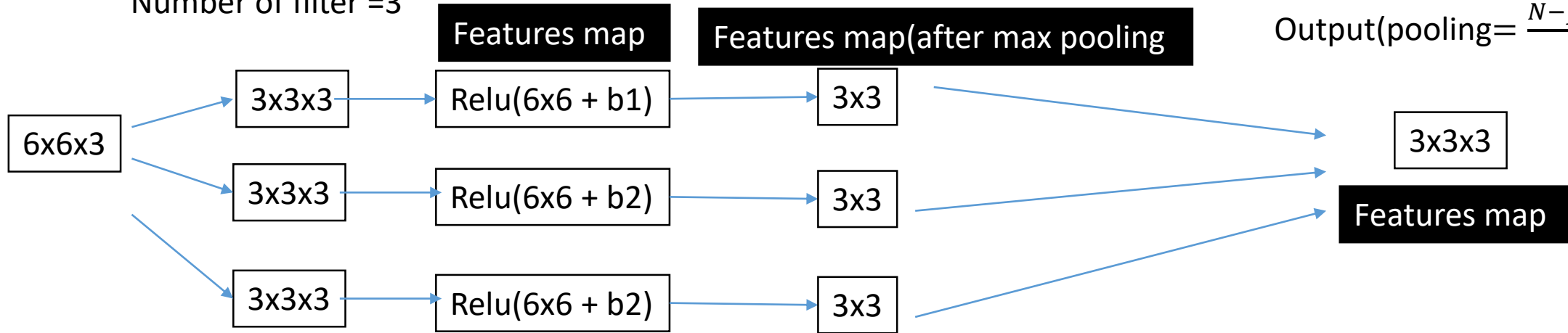


Padding = same
Stride = 1
Number of filter = 3

Max pooling (f=2x2)
Stride = 2

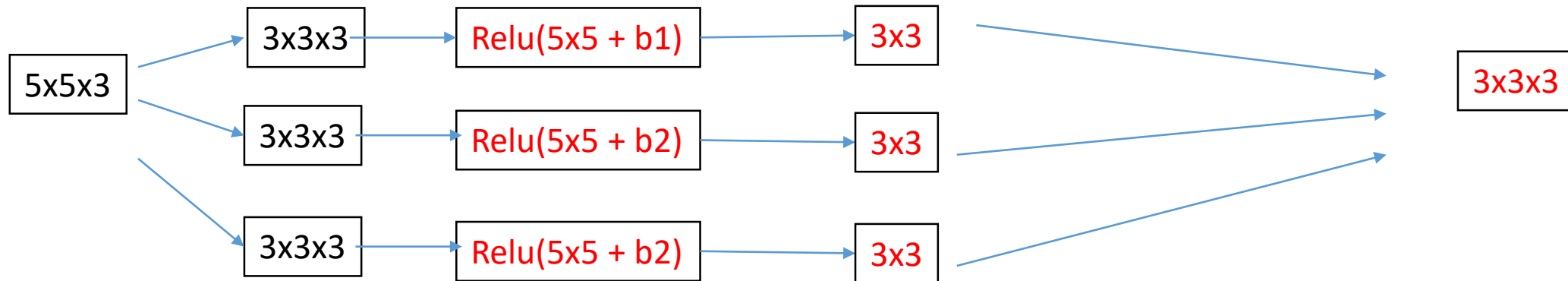
$$: \text{output(padding)} = \frac{N+2p-f}{s} + 1$$

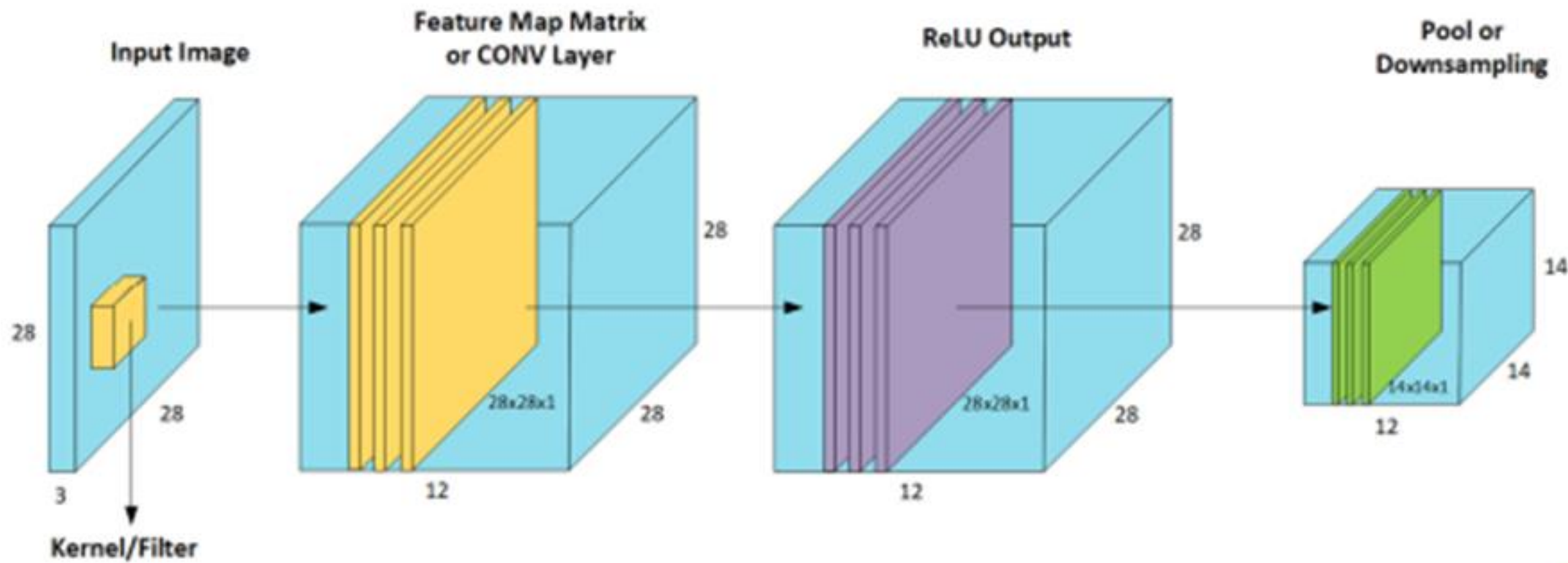
$$\text{Output(pooling)} = \frac{N-f(\text{pooling})}{s} + 1$$



Padding = same
Stride = 1
Number of filter = 3

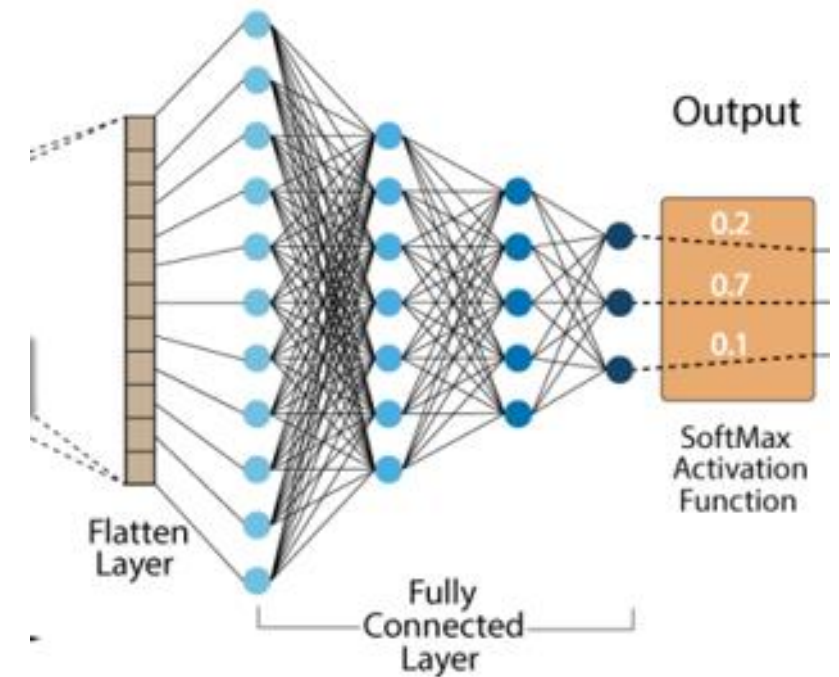
Max pooling (f=3x3)
Stride = 1



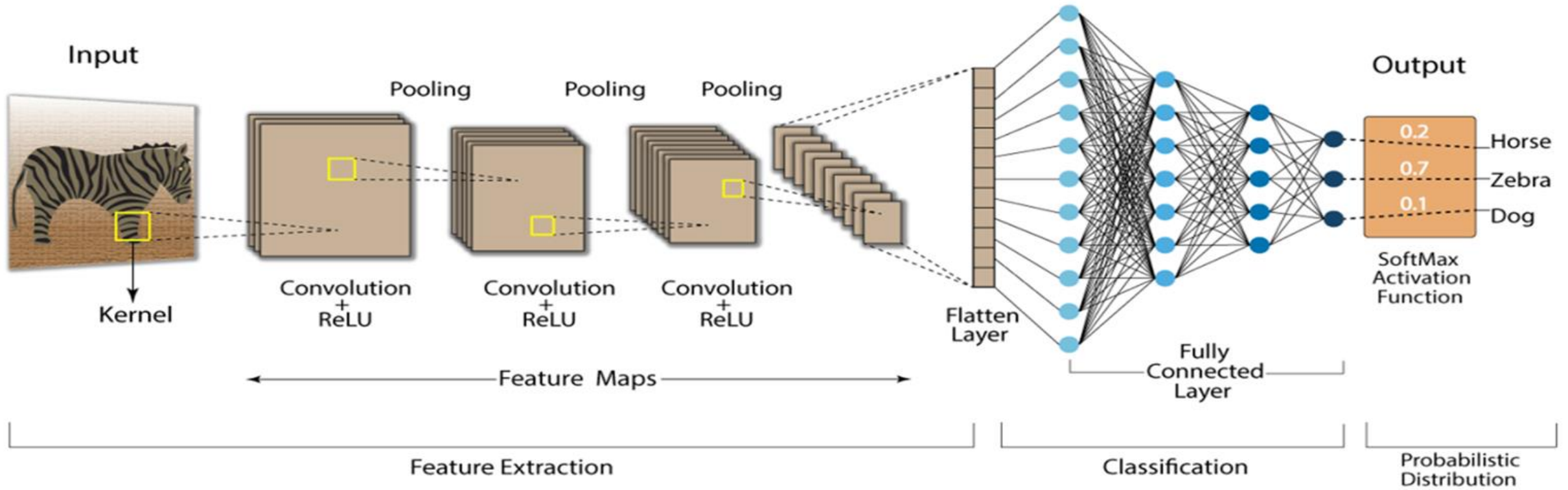


3: The Fully Connected Layer

- The activation function used to produce these probabilities is **the Soft Max Function** as it turns the outputs of the FC layer (last layer) into probabilities.
- The FC Layer outputs the class probabilities, where each class is assigned a probability.
- All probabilities must sum to 1, e,g (0.2, 0.5, 0.3)



Convolution Neural Network (CNN)



Summary

1 The raw image is fed to the convolutional layer, which is a set of kernel filters that slide over the image to extract features.

2 The convolutional layer has the following attributes that we need to configure:

```
from keras.layers import Conv2D
model.add(Conv2D(filters=16, kernel_size=2, strides='1',
padding='same', activation='relu'))
```

-`filters` is the number of kernel filters in each layer (the depth of the hidden layer).

-`kernel_size` is the size of the filter (aka kernel). Usually 2, or 3, or 5.

-`strides` is the amount by which the filter slides over the image. A `strides` value of 1 or 2 is usually recommended as a good start.

-`padding` adds columns and rows of zero values around the border of the image to reserve the image size in the next layer.

-`activation` of `relu` is strongly recommended in the hidden layers.

3 The pooling layer has the following attributes that we need to configure:

```
from keras.layers import MaxPooling2D
model.add(MaxPooling2D(pool_size=(2, 2), strides = 2))
```

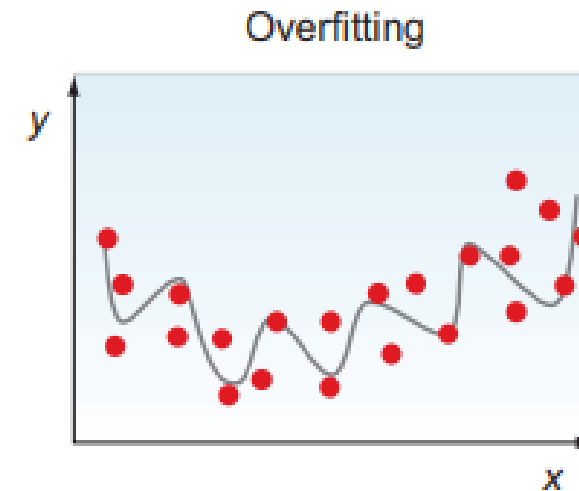
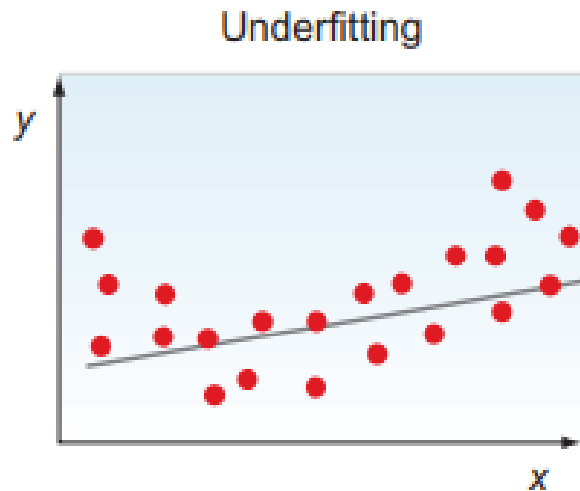
number of parameters

- number of parameters = filters $\times$ kernel size $\times$ depth of the previous layer + number of filters (for biases)
- Let's apply this equation in an example. Suppose we want to calculate the parameters at the second layer of the previous mini project. Here is the code for CONV_2 again:
 - `model.add(Conv2D(64, (3, 3), strides=1, padding='same', activation='relu'))`
- Since we know that the depth of the previous layer is 32, then (ex :6x6x32)
- $\Rightarrow \text{parameters} = 64 \times 3 \times 3 \times 32 + 64 = 18,496$

Adding dropout layers to avoid overfitting

Underfitting is as the name implies: the model fails to fit the training data. This happens when the model is too simple to fit the data: for example, using one perceptron to classify a nonlinear dataset.

Overfitting, on the other hand, means fitting the data too much: memorizing the training data and not really learning the features. This happens when we build a super network that fits the training dataset perfectly (very low error while training) but fails to generalize to other data samples that it hasn't seen before.



What is a dropout layer?

- A dropout layer is one of the most commonly used layers to prevent overfitting.
- Dropout turns off a percentage of neurons (nodes) that make up a layer of your network .

This percentage is identified as a hyper parameter that you tune when you build your network. By “turns off,” I mean these neurons are not included in a particular forward or backward pass.

It may seem counterintuitive to throw away a connection in your network, but as a network trains, some nodes can dominate others or end up making large mistakes. Dropout gives you a way to balance your network so that every node works equally toward the same goal, and if one makes a mistake, it won't dominate the behaviour of your model. You can think of dropout as a technique that makes a network resilient; it makes all the nodes work well as a team by making sure no node is too weak or too strong

